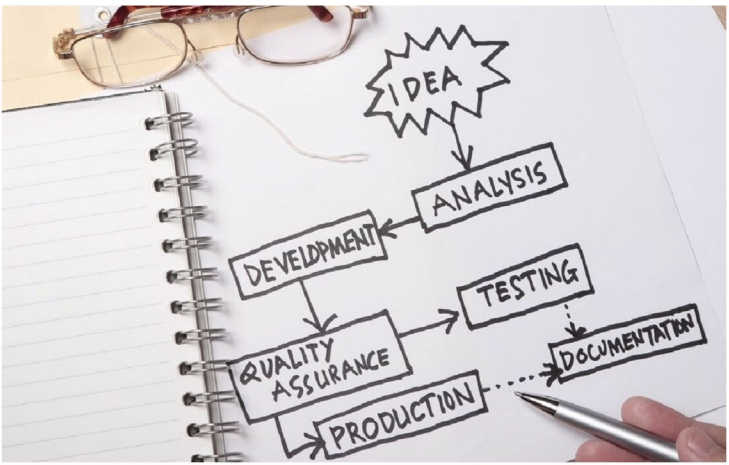
Reordering C Structure Alignment

The memory footprint of C programs can be reduced by manually repacking C structure declarations. This article is about reordering the structure members in decreasing lengths of variables.



Computer memory is word-addressable. For every instruction, the CPU fetches one word data from the memory. This behaviour affects how basic C data types will be aligned in memory. For example, every short (2B) will start on an address divisible by 2; similarly, int(4B) or long(8B) start on addresses divisible by 4 and 8, respectively. But char(1B) can start on any address. This happens due to *self-alignment* by the processor. Be it X86, ARM or PPC—all display this behaviour.

The following example will explain self-alignment and its advantages. In this example, the size of the pointer is assumed to be 4B. Here, we will use a user-defined struct data type.

```
struct ESTR
{
     char c;          /*1B*/
     unsigned int* up;    /*4B*/
     short x;         /*2B*/
     int v;           /*4B*/
     char k;          /*1B*/
};
```

You may think this structure will take 12B memory space. But if you print *sizeof(struct ESTR)* you will find the result to be 20B. This is because of self-alignment. This structure will be placed in memory in the following way:

BYTE1	BYTE2	BYTE3	BYTE4
C			
Up	up	up	up
x	x		
v	v	v	v
k			

So, you can see all the variables are starting on word boundaries. It is good practice to explicitly mention the *padding* so that no places are left with junk values. With padding, the structure resembles the following:

```
struct ESTR
{
     char c;          /*1B*/
     char pad1[3];
```